

Notes de référence pour l'entretien

Points clés et arguments à connaître pour l'oral

Document vivant : il s'enrichit au fil du projet, à chaque notion importante à retenir pour la présentation.

Table des matières

1	Pourquoi ne pas utiliser une IA de génération d'images	2
1.1	Le choix retenu	2
1.2	Pourquoi c'est le bon choix pour ce test	2
1.3	Le principe	3
1.4	Comment ajouter une vraie dimension « IA image » (pour aller plus loin)	3
1.5	En résumé	3
2	Un RAG aurait-il sa place dans ce projet ?	3
2.1	Le verdict en une phrase	3
2.2	Rappel : ce qu'est un RAG	3
2.3	Pourquoi ce n'est pas nécessaire pour le test	4
2.4	Où le RAG apporterait une vraie valeur (en V2)	4
2.5	La nuance d'ingénieur (à connaître)	4
2.6	Le lien avec la feuille de route de Gefradis	4
3	Comment l'application respecte toute la charte	4
3.1	Deux familles de règles, deux endroits	4
3.2	La charte visuelle : dans les gabarits et le code (déterministe)	4
3.3	Une seule police, une taille adaptée à chaque format	5
3.4	La charte éditoriale : dans le prompt système	5
3.5	La garantie architecturale	5
3.6	En toute transparence : les évolutions possibles	5
4	Résumer la charte dans le prompt fait-il perdre du contexte ?	6
5	Le prompt système ne risque-t-il pas de devenir trop lourd ?	6
5.1	Pourquoi le risque reste maîtrisé	6
5.2	Le vrai risque, et la parade	7
5.3	Le lien avec le RAG	7

6 Pourquoi Claude ne génère pas le HTML/CSS	7
6.1 Deux généricités à distinguer	7
6.2 Les risques si l'IA génère le HTML/CSS	7
6.3 Vérifié en pratique : comparaison A / B / C	8
6.4 La bonne façon d'être générique	8
7 Plan de tests (couverture des cas)	8
7.1 Type montant (€)	9
7.2 Type pourcentage (%)	10
7.3 Type avantage (non chiffré)	10
7.4 Cas limite : brief réaliste et imparfait (robustesse)	11
8 Résultats des tests	11
8.1 Bilan	12
8.2 Anomalies détectées et corrigées	12
9 Déploiement en production sur Google Cloud	12
9.1 L'architecture de production	12
9.2 Deux vrais problèmes rencontrés et résolus	13
9.3 Les commandes clés	13
9.4 Industrialisation : un pipeline CI/CD	14
9.5 Un arbitrage assumé : API directe plutôt que Vertex AI	14

1. Pourquoi ne pas utiliser une IA de génération d'images

1.1. Le choix retenu

L'outil n'utilise **pas** d'IA générative d'images (DALL-E, Midjourney, Stable Diffusion) pour créer les visuels. C'est le **code**, via Playwright, qui assemble des éléments graphiques *existants* (textes, couleurs, logo, gabarits). L'IA (Claude) intervient sur le **texte et la structure**, pas sur le dessin.

1.2. Pourquoi c'est le bon choix pour ce test

- **Respect de la marque (brand safety)** : une IA d'image est incapable de reproduire fidèlement une fenêtre ou une porte Gefradis précise. Elle invente des formes, et déformerait aussi le logo.
- **Le texte** : les modèles d'images écrivent encore mal le texte (fautes de frappe, lettres déformées). Or nos bannières sont essentiellement du texte.
- **Les dimensions exactes** : générer une image au format exact (320x50, 160x600...) avec une IA donne souvent des visuels écrasés ou rognés.

1.3. Le principe

On utilise l'IA là où elle excelle (la réflexion, le copywriting, la structure) et le code là où il est imbattable (la précision géométrique et le respect de la charte). **Claude pour le texte, Playwright pour l'assemblage.**

1.4. Comment ajouter une vraie dimension « IA image » (pour aller plus loin)

Deux approches possibles, sans casser l'architecture :

Option A : l'IA génère le fond d'ambiance. L'utilisateur demande un fond personnalisé via une API d'image (DALL-E 3, Stable Diffusion). Exemple de prompt envoyé par l'application : *« Une maison moderne lumineuse avec de grandes baies vitrées, style épuré, flou d'arrière-plan, tons bleus et blancs sourds. »* Le code récupère cette image, la place en arrière-plan, et Playwright superpose par-dessus le logo Gefradis parfait, les textes nets de Claude et le bouton d'action.

Option B : l'IA choisit le meilleur asset. Si Gefradis fournit une banque d'images (`fenetre_pvc.png`, `porte_aluminium.png`, `volet_roulant.png`), Claude analyse le brief et renvoie le bon fichier dans son JSON, par exemple `"asset_a_utiliser": "fenetre_pvc.png"`. Le code va alors chercher automatiquement le bon visuel. C'est une utilisation intelligente de la gestion des assets.

1.5. En résumé

Le projet contient bien une dimension IA : un LLM (Claude) pour l'adaptation intelligente des messages et la structure du contenu. La génération d'image pure est volontairement écartée pour garantir la fidélité au produit et au logo.

Phrase à dire à l'oral

« J'ai exclu la génération d'image pure par IA pour garantir que le produit Gefradis et le logo soient reproduits à 100% sans hallucination. En revanche, l'IA intervient intelligemment dans la conceptualisation textuelle et contextuelle de la bannière. Si besoin, on peut tout à fait connecter une API de génération d'image (par exemple DALL-E) pour concevoir uniquement les fonds d'ambiance. »

2. Un RAG aurait-il sa place dans ce projet ?

2.1. Le verdict en une phrase

Pas de RAG pour le rendu du test : le cœur de la valeur IA, c'est le **workflow LLM** (du brief vers le copy, puis vers la mise en page), pas la recherche documentaire. Le RAG devient pertinent **en V2**, quand la base de connaissances grandit.

2.2. Rappel : ce qu'est un RAG

Un RAG (*Retrieval-Augmented Generation*) va chercher, dans une base de documents, les passages utiles, puis les injecte dans le contexte du modèle au moment de répondre. Il sert quand la connaissance est **trop volumineuse** ou **trop changeante** pour tenir dans le prompt.

2.3. Pourquoi ce n'est pas nécessaire pour le test

- La charte actuelle (ton, couleurs, arguments) tient en quelques phrases : elle va directement dans le **prompt système**. Un RAG par-dessus serait de la sur-ingénierie.
- Le cas d'usage est direct : un brief en entrée, un kit en sortie. Le LLM seul suffit.
- Savoir *ne pas* ajouter un outil prématuré est aussi une compétence d'ingénieur.

2.4. Où le RAG apporterait une vraie valeur (en V2)

- **Sélection automatique des visuels** : avec une banque de photos produit (fenêtres, portes, volets) décrites par des métadonnées, retrouver le bon visuel pour un brief donné. C'est l'évolution naturelle de l'Option B.
- **Réutilisation des campagnes passées** : retrouver les opérations similaires déjà réalisées, pour garder une cohérence et inspirer le copy.
- **Charte et guidelines détaillées qui évoluent** : quand la documentation marketing devient volumineuse et change souvent, on la met à jour dans la base, sans toucher au code.

2.5. La nuance d'ingénieur (à connaître)

- **C'est une question d'échelle** : le RAG se justifie quand la base est grande et évolutive ; sur une petite charte fixe, le prompt suffit.
- **Retrouver un asset n'exige pas forcément un RAG complet** : pour une banque modeste, une simple correspondance par métadonnées, ou un choix par le LLM dans une liste, suffit. La base vectorielle (*embeddings*) se justifie sur un grand volume.

2.6. Le lien avec la feuille de route de Gefradis

Gefradis développe déjà un RAG pour la connaissance produit (accompagnement client). L'outil de bannières pourrait, en V2, se brancher sur cette même base. C'est cohérent avec leur volonté de multiplier les projets IA.

Phrase à dire à l'oral

« Pour le test, je n'ai pas mis de RAG : le cœur IA, c'est le workflow LLM (du brief vers le copy, puis vers la mise en page), et la charte tient dans le prompt. En V2, un RAG aurait une vraie valeur pour sélectionner automatiquement les visuels produit, réutiliser les campagnes passées et exploiter une charte détaillée qui évolue. C'est une question d'échelle : on l'ajoute quand la base documentaire devient assez grande pour le justifier. Il pourrait d'ailleurs se brancher sur le RAG produit que vous développez déjà. »

3. Comment l'application respecte toute la charte

3.1. Deux familles de règles, deux endroits

La charte se divise en deux types de règles, respectées à deux endroits distincts. C'est le point central de l'architecture.

3.2. La charte visuelle : dans les gabarits et le code (déterministe)

Les règles visuelles sont **codées en dur** dans le gabarit HTML/CSS (composable/backend/bannières/) :

- **Couleurs exactes** (#253081, #FFD52B, #FFFDE5) déclarées en variables CSS.
- **Police Cabin, graisse 700 maximum** : on s'interdit d'aller au-delà, même si la police variable le permettrait.
- **Coins droits** : aucun arrondi.
- **Structure de marque** : carte bleue, titre blanc, chiffre dans le bloc jaune, figée dans le gabarit.
- **Dimensions exactes** : fixées par Python et Playwright pour chaque format.

3.3. Une seule police, une taille adaptée à chaque format

La charte impose la **police** (Cabin) et la **hiérarchie** des éléments, mais **pas** les tailles exactes : un kit qui va du 320×50 au 2560×280 ne peut pas utiliser la même taille de texte partout. Le dimensionnement est donc une décision d'ingénierie, conçue pour rester cohérente :

- Chaque format calcule une **taille de base** proportionnelle à ses dimensions (par exemple proportionnelle à sa plus petite dimension).
- Tous les éléments (titre, prix, bouton, paliers) sont des **multiples de cette base**, avec une hiérarchie cohérente d'un format à l'autre.
- Le contenu est enfin **mis à l'échelle** pour tenir exactement dans le format, sans débordement.

Résultat : la **police** est la même partout, la **taille absolue** s'adapte à chaque format, et les **proportions** restent constantes, ce qui assure la cohérence de marque du petit display au grand bandeau.

Phrase à dire à l'oral

« Toutes les bannières utilisent la même police, Cabin, et le même système de proportions. La taille, elle, est calculée pour chaque format : une base proportionnelle aux dimensions, puis les éléments en multiples de cette base, et enfin une mise à l'échelle. La charte fixe la police et la hiérarchie, pas les tailles exactes : cette adaptation est une décision de conception pour rester cohérent sur tous les formats. »

3.4. La charte éditoriale : dans le prompt système

Ce que les gabarits ne peuvent pas garantir, c'est la **voix** du texte. Le prompt système porte donc le contexte de marque (charte 4 : fabricant français, Dunkerque depuis 1996, direct usine, sur-mesure) et le ton (charte 6 : professeur ou mentor, précis, transparent), pour que le copy sonne Gefradis.

3.5. La garantie architecturale

Claude génère **uniquement du JSON** (le texte et le choix de variante), **jamais le HTML/CSS**. Il remplit seulement les emplacements texte. Il lui est donc **physiquement impossible** d'introduire une couleur, une police ou une forme hors charte. La créativité de l'IA reste enfermée dans des briques déjà conformes.

3.6. En toute transparence : les évolutions possibles

Le cœur est en place (logo intégré selon le fond, palette de la charte, photos produit détournées, gabarit composable) et l'application est **déployée**. Restent des pistes d'évolution (V2) : la **sélection des visuels par métadonnées** (au lieu de « première image du dossier ») ; un **contrôle**

qualité automatique (vérifications en code, puis relecture IA optionnelle) ; et le champ **1.3 pilotant la mise en page** (l'utilisateur décrit où placer les éléments, l'IA renvoie des options balisées que le gabarit applique).

Phrase à dire à l'oral

« La charte est respectée à deux niveaux : le visuel (couleurs, police, formes, dimensions) est codé en dur dans les gabarits, donc déterministe ; l'éditorial (le ton, le positionnement) est porté par le prompt système. Et comme Claude ne génère que du texte, jamais le code visuel, il lui est impossible de sortir de la charte graphique. »

4. Résumer la charte dans le prompt fait-il perdre du contexte ?

Résumer le contexte de marque pour le prompt système ne fait pas perdre d'information importante, pour quatre raisons.

- **Seule la charte éditoriale va dans le prompt.** La charte visuelle n'y va pas du tout (elle est dans les gabarits). Résumer le prompt ne perd donc aucune règle visuelle : Claude n'a pas besoin de connaître les codes couleur, il ne dessine pas.
- **Résumer n'est pas perdre.** On garde les consignes opérationnelles (descripteurs de ton exacts, faits de marque clés) et on enlève seulement le remplissage explicatif.
- **Un prompt court et ciblé est souvent meilleur qu'un prompt gonflé.** Trop de texte dilue l'attention du modèle et coûte plus cher. Le but n'est pas le maximum de contexte, mais le bon contexte.
- **La charte est petite.** Le contexte de marque et le ton tiennent en un ou deux paragraphes : le résumé est léger.

Garde-fous si le ton dérive : garder des consignes concrètes (à faire, à éviter) ; ajouter un ou deux exemples de bon copy dans le prompt (un exemple transmet le ton mieux qu'une description) ; itérer sur la sortie réelle et réinjecter la nuance manquante. C'est du prompt engineering normal.

Phrase à dire à l'oral

« Résumer la charte pour le prompt ne fait pas perdre de contexte : seule la partie éditoriale y va, la partie visuelle est dans le code. Et un prompt court et ciblé est souvent plus efficace qu'un prompt gonflé, qui dilue l'attention du modèle. Au besoin, j'ajoute des exemples et j'itère sur la sortie. »

5. Le prompt système ne risque-t-il pas de devenir trop lourd ?

Le constat : à chaque champ que Claude doit produire (offre, thème transverse, paliers, date...), on ajoute une consigne. Le prompt pourrait gonfler. C'est une vraie question de maintenabilité.

5.1. Pourquoi le risque reste maîtrisé

- **Le schéma de l'outil porte la spec.** Avec le *tool use*, chaque champ a sa **description**. La majeure partie des consignes spécifiques à un champ vit donc dans le **schéma**, pas dans le prompt en prose.
- **Le périmètre est borné.** Le nombre de champs rédigés par Claude est fixé par le produit (offre, catégorie, thème transverse, paliers, date) : il **plafonne**, il ne croît pas à l'infini. L'image, le bouton et le fond, eux, viennent du formulaire, pas du prompt.

- **Coût et performance.** Le system prompt reste petit face à la capacité du modèle, et il peut être **mis en cache** (prompt caching) : les appels répétés ne le re-paient quasiment pas.

5.2. Le vrai risque, et la parade

Le danger d'un prompt qui gonfle, c'est de **diluer l'attention** du modèle et de devenir dur à maintenir, surtout quand une consigne est écrite **deux fois** (dans le prompt ET dans la description du champ). La règle de rangement :

- **Prompt en prose** = règles **générales et transversales** (rôle, marque, ton, « titre court », « n'invente aucun chiffre », « réponds via l'outil »).
- **Description des champs** (schéma) = consignes **spécifiques** à chaque champ.

Ainsi, ajouter une fonctionnalité revient à ajouter **une description de champ**, sans gonfler le prompt : il reste stable et lisible.

5.3. Le lien avec le RAG

Tant que la connaissance (charte éditoriale, consignes) tient proprement dans le prompt et le schéma, **un RAG est inutile** (sur-ingénierie). Le RAG devient pertinent le jour où cette connaissance devient **volumineuse et évolutive** (guidelines détaillées qui changent souvent, base de campagnes passées) : on **l'externalise** alors dans une base interrogée à la demande (V2). C'est une question d'échelle.

Phrase à dire à l'oral

« Le prompt ne gonfle pas tant qu'on met chaque consigne au bon endroit : les règles générales dans le prompt, les consignes par champ dans le schéma de l'outil (tool use). Ajouter une fonctionnalité, c'est ajouter une description de champ, pas une ligne de prompt. Et le jour où la connaissance devient trop grosse ou trop changeante pour tenir dans le prompt, on l'externalise dans un RAG : c'est une question d'échelle. »

6. Pourquoi Claude ne génère pas le HTML/CSS

6.1. Deux généricités à distinguer

L'outil doit être générique, car il sert des **opérations commerciales** variées. Mais il faut distinguer :

- la généricité du **contenu** (n'importe quelle offre, n'importe quelle opération) : **nécessaire**, et c'est le rôle de Claude ;
- la liberté **visuelle** (mise en page et CSS au gré de l'IA) : **risquée**, et inutile.

Laisser Claude produire le HTML/CSS, c'est lui donner la liberté visuelle. On s'en abstient volontairement.

6.2. Les risques si l'IA génère le HTML/CSS

- **Fidélité à la charte : plus aucune garantie.** L'IA pourrait sortir une couleur hors palette, une autre police, une graisse interdite, des coins arrondis. Avec des gabarits fixes, c'est impossible.
- **Dimensions exactes : cassées.** Rien ne garantit qu'un HTML généré remplisse pile le format sans déborder.

- **Fiabilité.** Un JSON est validé contre un schéma (tool use) ; du HTML libre peut être tronqué (limite de tokens) ou mal formé, et le rendu casse.
- **Cohérence du kit.** Les formats d'une même campagne risqueraient d'être incohérents entre eux.
- **Assets.** Logo, images produit, police, accents : c'est le code qui gère ces chemins avec précision.
- **Coût et débogage.** Plus de tokens, et un HTML généré est très dur à tester et à maintenir.

6.3. Vérifié en pratique : comparaison A / B / C

Pour ne pas en rester à la théorie, les trois architectures ont été construites et comparées sur les mêmes briefs :

- **A — gabarits figés + JSON** : un gabarit par type de bannière ; Claude ne fournit que le texte structuré. Rendu très régulier et fidèle à la charte, un peu rigide quand le contenu est riche (paliers, image).
- **B — composable + JSON** : un gabarit unique et adaptatif ; Claude fournit le texte ET propose la disposition. Même fidélité de charte que A (les briques restent figées), mais une mise en page qui s'adapte mieux au contenu chargé.
- **C — libre / full-LLM** : Claude génère lui-même tout le HTML/CSS, la charte ne lui étant que décrite.

La version C a **confirmé empiriquement** les risques ci-dessus : des résultats **imprévisibles** (logo surdimensionné, mise en page qui varie d'un format à l'autre, écarts de charte possibles), en plus d'être **lente et coûteuse** (un appel au modèle par format). C'est précisément ce qui justifie de garder le JSON structuré. On **écarte donc C**, et le vrai choix se joue entre **A et B**, qui garantissent toutes deux la charte (B ajoutant une mise en page adaptative).

Phrase à dire à l'oral

« J'ai même implémenté la version full LLM pour la confronter aux deux autres : elle produit des résultats imprévisibles et coûteux, ce qui valide concrètement le choix du JSON structuré. La décision finale se joue donc entre les gabarits figés et l'approche composable, qui garantissent toutes deux la charte. »

6.4. La bonne façon d'être générique

Claude reste cantonné à un **JSON structuré** ; le code rend ce JSON dans des gabarits flexibles. Pour l'offre, Claude indique un **type** (montant, pourcentage ou texte) et le code choisit le rendu adapté, **toujours à la charte**. On obtient la genericité du *contenu* sans sacrifier la fidélité visuelle, les dimensions et la fiabilité.

Phrase à dire à l'oral

« Je contrains volontairement l'IA au texte structuré : c'est le code qui garantit la charte, les dimensions exactes et la cohérence du kit. Si je laissais l'IA générer le HTML, je perdrais ces garanties pour rien : la genericité dont j'ai besoin est dans le contenu, pas dans la liberté visuelle. »

7. Plan de tests (couverture des cas)

Les briefs ci-dessous sont rédigés **comme le ferait l'équipe marketing** (en prose, avec un contexte), et NON taillés pour les fonctionnalités : ils servent à **vérifier que l'outil se comporte**

bien sur des cas réels. Rappel du modèle de saisie : l'**offre** (montant / pourcentage / avantage, paliers, date) est détectée dans 1.1 et 1.2 ; l'**image**, le **bouton** et la **couleur de fond** sont des choix du formulaire. Quel que soit son type, l'offre est affichée comme un **composant intégré à la composition** (le bloc jaune de la charte), dimensionné et placé selon le format. Chaque cas donne le brief (1.1/1.2/1.3), les réglages du formulaire, puis *ce qu'on vérifie*.

7.1. Type montant (€)

M1 • Lancement gamme portes (catégorie, prix d'appel)

- **1.1 Mécanique** Nous lançons notre nouvelle collection de portes d'entrée PVC sur-mesure, fabriquées en France. Prix de lancement à partir de 639,85 € la porte (pose en supplément), valable jusqu'à la fin du mois, dans la limite des coloris en stock.
- **1.2 Message** Opération « Vos portes d'entrée au prix d'usine ». Donner envie de découvrir la nouvelle collection et rassurer sur la fabrication française et le sur-mesure.
- **1.3 Direction créative** Ton haut de gamme et chaleureux, insister sur le sur-mesure.
- **Formulaire** Image : avec ; Bouton : « Découvrir la collection » ; Fond : Blanc bleuté (#E7E9F4).
- **On vérifie** La catégorie **Portes** est déduite ; offre de type **montant** (« à partir de 639,85 € ») ; l'image de portes apparaît sur les pubs et le bandeau (l'en-tête reste typographique, sans image produit) ; le bouton est présent ; la date « jusqu'à la fin du mois » s'affiche ; logo bleu (fond clair) ; le titre est une accroche, pas la répétition du prix.

M2 • Black Friday (transverse, paliers, événement)

- **1.1 Mécanique** Black Friday sur tout le catalogue : remise progressive selon le montant de la commande, 100 € offerts dès 1000 € d'achat HT et jusqu'à 500 € offerts sur les plus gros paniers. Du 24 au 30 novembre, une semaine seulement.
- **1.2 Message** Black Friday Gefradis : message événementiel et urgent, des remises exceptionnelles sur toute la gamme. Jouer la rareté (offre limitée dans le temps).
- **1.3 Direction créative** Ambiance Black Friday, percutant.
- **Formulaire** Image : avec ; Bouton : « J'en profite » ; Fond : Bleu nuit (#040C4D).
- **On vérifie** Catégorie **transversale** ; le thème **Black Friday** est reconnu (visuel d'événement dans les pubs et à côté de la carte du bandeau) ; les **paliers** sont extraits (100 € / 500 €) ; l'urgence et les dates ressortent ; logo bleu-jaune (fond foncé).

M3 • Rénovation fenêtres (montant par fenêtre, sobre, sans image)

- **1.1 Mécanique** Offre rénovation sur nos fenêtres PVC double vitrage : 100 € offerts par fenêtre dès trois fenêtres commandées, cumulables sur l'ensemble de la commande.
- **1.2 Message** Encourager la rénovation complète en récompensant les commandes multiples. Ton pédagogique, insister sur l'isolation thermique et les économies d'énergie.
- **1.3 Direction créative** Sobre et élégant ; on veut que le message et le chiffre parlent d'eux-mêmes.
- **Formulaire** Image : sans ; Bouton : aucun ; Fond : Gris clair (#F3F4F6).
- **On vérifie** Catégorie **Fenêtres**, type montant ; pubs **typographiques** (sans image), sans bouton ; le rendu reste lisible et équilibré sans image ; le bandeau garde sa photo d'ambiance.

M4 • Gros projets = Grosses remises (transverse, paliers + date)

- **1.1 Mécanique** Opération « Gros projets = Grosses remises » sur tout le catalogue : 100 € offerts dès 1000 € HT, et 350 € offerts dès 3000 € HT. Valable jusqu'au 30 juin 2026.
- **1.2 Message** Récompenser les gros chantiers (rénovation complète, plusieurs ouvertures) avec une remise qui grimpe par paliers. Créer une opportunité avant la date limite.
- **1.3 Direction créative** Mettre en avant l'idée « plus votre projet est grand, plus vous économisez ».

- **Formulaire** Image : avec ; Bouton : « J'en profite » ; Fond : Bleu Gefradis (#253081).
- **On vérifie Deux paliers** {condition, valeur} extraits + une **date complète** ; sur les grands formats les paliers s'affichent en étiquettes, sur les petits l'offre résumée (la plus avantageuse) ; image + bouton présents ; le titre est une accroche distincte de l'offre.

7.2. Type pourcentage (%)

P1 • Déstockage fenêtres (-30%, avec image)

- **1.1 Mécanique** Déstockage de fin de série : -30 % sur une sélection de fenêtres PVC (coloris standard), remise immédiate, dans la limite des stocks.
- **1.2 Message** Promo dynamique et lisible ; mettre clairement le pourcentage en avant pour déclencher l'achat avant rupture.
- **1.3 Direction créative** Style promo, dynamique.
- **Formulaire** Image : avec ; Bouton : aucun ; Fond : Blanc bleuté (#E7E9F4).
- **On vérifie** Type **pourcentage** (-30 %), image de fenêtres ; le **-30 %** s'affiche comme l'offre, dans le **bloc jaune intégré à la composition** (même traitement que les montants et les avantages), bien lisible à côté de l'image.

P2 • French Days (transverse, thème inconnu, -20%)

- **1.1 Mécanique** Opération French Days : -20 % sur l'ensemble de nos menuiseries PVC, sans minimum d'achat. Du 1^{er} au 5 mai.
- **1.2 Message** Surfer sur l'événement French Days, valoriser la fabrication française, remise claire sur toute la gamme.
- **1.3 Direction créative** Esprit « made in France », aux couleurs de la marque.
- **Formulaire** Image : avec ; Bouton : aucun ; Fond : Bleu Gefradis (#253081).
- **On vérifie** Catégorie transversale ; le thème « French Days » est **inconnu** (absent du dossier) → **repli sur la promo par défaut** dans les pubs ; type pourcentage ; le -20 % s'affiche comme offre intégrée (bloc jaune) ; logo bleu-jaune.

P3 • Volets roulants motorisés (-25% sur la motorisation, sans image)

- **1.1 Mécanique** Offre saisonnière sur les volets roulants PVC motorisés : -25 % sur la motorisation pour toute commande de volets roulants neufs (hors pose).
- **1.2 Message** Promouvoir le confort au quotidien et les économies d'énergie ; inciter à demander un devis.
- **1.3 Direction créative** Ton incitatif, orienté confort.
- **Formulaire** Image : sans ; Bouton : « Demander un devis gratuit » ; Fond : Gris clair (#F3F4F6).
- **On vérifie** Catégorie Volets roulants, type pourcentage ; le **suffixe reste court** (« % ») et « sur la motorisation » va dans le titre ou le libellé (pas coincé dans le bloc jaune) ; bouton présent ; le **-25 %** s'affiche comme offre intégrée à la composition, lisible même sans image.

7.3. Type avantage (non chiffré)

A1 • Baies vitrées, pose offerte (avantage, avec image)

- **1.1 Mécanique** Opération sur nos baies vitrées coulissantes PVC sur-mesure : la pose est offerte pour toute commande d'une baie, sur rendez-vous.
- **1.2 Message** Lever le frein de la pose en mettant « pose offerte » en avant ; valoriser la luminosité et le confort.
- **1.3 Direction créative** Lumineux, valoriser l'espace de vie.
- **Formulaire** Image : avec ; Bouton : « Prendre rendez-vous » ; Fond : Blanc bleuté (#E7E9F4).

- **On vérifie** Catégorie Baies vitrées; type **avantage** (le bloc jaune affiche « Pose offerte », sans chiffre); image + bouton présents.

A2 • Livraison offerte (transverse, avantage, sans image)

- **1.1 Mécanique** Avantage permanent sur tout le catalogue : livraison offerte partout en France métropolitaine dès 1000 € d'achat.
- **1.2 Message** Rassurer sur la livraison, renforcer l'argument du prix d'usine sans intermédiaire; ton clair et professionnel.
- **1.3 Direction créative** Épuré, rassurant.
- **Formulaire** Image : sans; Bouton : aucun; Fond : Bleu nuit (#040C4D).
- **On vérifie** Catégorie transversale; type **avantage** (« Livraison offerte »); pubs typographiques sans image; bandeau `bandeau.png` + carte; logo bleu-jaune; le titre ne répète pas l'offre.

A3 • Portails, motorisation offerte (avantage, avec image)

- **1.1 Mécanique** Opération sur les portails PVC et aluminium sur-mesure : pour toute commande d'un portail, la motorisation est offerte (valeur jusqu'à 600 €).
- **1.2 Message** Valoriser le confort de la motorisation et le sur-mesure; « motorisation offerte » comme argument fort.
- **1.3 Direction créative** Confort moderne, sur-mesure.
- **Formulaire** Image : avec; Bouton : « Configurer mon portail »; Fond : Gris clair (#F3F4F6).
- **On vérifie** Catégorie Portails; type **avantage** (« Motorisation offerte »); image + bouton présents; le titre est une accroche distincte de l'offre.

7.4. Cas limite : brief réaliste et imparfait (robustesse)

R1 • Brief brouillon, comme dans la vraie vie

- **1.1 Mécanique** « Grosse opé de rentrée sur les fenêtres!! 100 € offerts dès 1000 € d'achat, et encore 100 € dès 1000 € (à révéifier avec la compta), valable tout le mois de septembre je crois. »
- **1.2 Message** Opération rentrée, ton dynamique.
- **1.3 Direction créative** (laissé vide)
- **Formulaire** Image : avec; Bouton : aucun; Fond : Bleu Gefradis (#253081).
- **On vérifie** L'outil reste **robuste** à un brief brouillon (fautes, seuils en doublon « 1000 € » deux fois, date vague) : il ne plante pas, déduit une catégorie et produit un kit cohérent. **Limite connue** : il ne signale pas l'incohérence des seuils, c'est précisément là qu'un **agent de relecture** apporterait de la valeur (détecter et demander confirmation).

Phrase à dire à l'oral

« Mes briefs de test sont écrits comme de vrais briefs marketing, pas calibrés sur mes fonctionnalités : ils servent à vérifier que l'outil se comporte bien sur des cas réels. Je couvre les trois types d'offre, les paliers et les dates, les opérations catégorie et transverse, le repli par défaut, les fonds clairs et foncés, et même un brief volontairement brouillon pour tester la robustesse. Tester, c'est du travail d'ingénieur. »

8. Résultats des tests

Les neuf premiers briefs (M1 à A3) ont été exécutés sur l'ensemble des formats sur la version A. Après une série de corrections, tous les rendus sont conformes aux résultats attendus. La campagne a surtout permis de **détecter et corriger plusieurs anomalies**, ce qui illustre une

vraie démarche de test. Le dixième brief (M4, plusieurs paliers et date de validité) a montré les **limites des gabarits figés** de la version A : avec un contenu plus riche, les mises en page rigides débordaient. C'est ce constat qui a motivé l'exploration des architectures B (composable) et C (libre), puis leur comparaison. La version **B (composable) a finalement été retenue, finalisée et déployée en ligne** : elle exploite tout l'espace et gère sans débordement les contenus riches (paliers, date) qui mettaient les gabarits figés de A en difficulté.

8.1. Bilan

- **Couvert et validé** : déduction de la catégorie, du type d'offre (montant / pourcentage / avantage), des paliers, de la date et du thème transverse (Black Friday) ; image, bouton et couleur de fond choisis au formulaire ; logo selon le fond ; bandeaux catégorie et transverse.
- **Repli par défaut** : une opération transverse sans thème connu (French Days) utilise bien la *promo par défaut*.

8.2. Anomalies détectées et corrigées

- **Logo qui chevauchait le titre** (formats portrait illustrés) : en mode colonne, le logo en position absolue passait sur le titre. *Correctif* : en colonne, le logo est placé dans le flux, au-dessus du titre.
- **Header mobile (426x575) déformé** : l'offre débordait, la taille de base étant calculée sur la hauteur. *Correctif* : base sur la plus petite dimension, et l'offre passe à la ligne si besoin.
- **Image du bandeau coupée** : le *cover* rognait l'image sur une bande très large. *Correctif* : image posée à pleine hauteur puis répétée pour remplir (catégorie et transverse).
- **Anciens fichiers dans le zip** : le dossier de sortie accumulait les formats des générations précédentes. *Correctif* : il est vidé avant chaque génération.
- **Promo par défaut absente (P2)** : sur une transverse sans thème, l'IA mettait parfois illustrer à false. *Correctif* : détection de l'illustration renforcée (par le sens : « un visuel », « une image »).
- **Signe des pourcentages** : « 20% » s'affichait sans le « - ». *Correctif* : le « - » est ajouté selon le sens (présent pour « Jusqu'à 20% », absent pour « Économisez 20% »).

Phrase à dire à l'oral

« J'ai mené une campagne de tests structurée : neuf briefs couvrant les trois types d'offre et tous les formats. Elle a révélé des anomalies concrètes (chevauchement du logo, débordement sur mobile, image de bandeau rognée, signe des remises...) que j'ai corrigées une à une. C'est cette boucle test, diagnostic, correction qui donne confiance dans le rendu final. »

9. Déploiement en production sur Google Cloud

L'application n'est pas qu'un prototype local : elle est **déployée et accessible en ligne** sur Google Cloud, à une URL publique.

9.1. L'architecture de production

- **Image Docker** (Python + Playwright/Chromium + police Cabin) : l'application empaquetée, identique en local et en ligne, stockée dans **Artifact Registry**.
- **Cloud Run** exécute des conteneurs à partir de cette image. Il **scale automatiquement** (plusieurs instances en cas de trafic, **zéro** quand c'est inactif, donc 0 € au repos).

- **Secret Manager** contient la clé de l'API Claude : jamais dans l'image ni dans le code, injectée à l'exécution.
- **Cloud Storage** stocke les bannières générées, dans un bucket partagé monté comme dossier de sortie de l'application.

9.2. Deux vrais problèmes rencontrés et résolus

Le déploiement n'a pas été un simple « clic » : deux difficultés concrètes, typiques du cloud, ont dû être comprises et corrigées.

- **Image refusée par Cloud Run (« Container import failed »)**. Par défaut, le constructeur d'images (BuildKit) ajoute des *attestations* qui transforment l'image en « manifest list », que Cloud Run ne sait pas importer. *Correctif* : reconstruire une image simple (`--provenance=false`).
- **Images manquantes dans l'aperçu en ligne**. Cloud Run lance plusieurs instances ; les bannières écrites sur le disque d'une instance n'existaient pas sur les autres. En chargeant les 17 visuels, certains tombaient sur une instance « vide » → image manquante. C'est le piège du **système de fichiers éphémère et par-instance** de Cloud Run. *Correctif* : stocker les sorties dans **Cloud Storage** (bucket partagé), visible par toutes les instances.

9.3. Les commandes clés

1. Activer les services

```
gcloud services enable run.googleapis.com cloudbuild.googleapis.com \
  artifactregistry.googleapis.com secretmanager.googleapis.com
```

2. Cle API dans Secret Manager

```
printf '%s' "CLE" | gcloud secrets create anthropic-api-key \
  --data-file=- --replication-policy=automatic
```

3. Construire et pousser l'image (image simple, sans attestation)

```
docker build --provenance=false --platform linux/amd64 -t IMG .
docker push IMG
```

4. Deployer sur Cloud Run (cle + bucket + scaling)

```
gcloud run deploy gefradis-kits --image IMG --region europe-west9 \
  --memory 2Gi --cpu 2 --execution-environment gen2 --allow-unauthenticated \
  --set-secrets ANTHROPIC_API_KEY=anthropic-api-key:latest \
  --add-volume=name=kitvol,type=cloud-storage,bucket=BUCKET \
  --add-volume-mount=volume=kitvol,mount-path=/app/composable/output
```

Phrase à dire à l'oral

« J'ai conteneurisé l'application avec Docker, puis je l'ai déployée sur Cloud Run : l'image est dans Artifact Registry, la clé API dans Secret Manager, et les bannières générées dans Cloud Storage. Cloud Run scale tout seul et descend à zéro quand c'est inactif. J'ai aussi résolu deux vrais problèmes de cloud : une image refusée à cause des attestations BuildKit, et le système de fichiers éphémère par-instance, corrigé en passant par un bucket Cloud Storage partagé. »

9.4. Industrialisation : un pipeline CI/CD

Au départ, je redéployais **à la main** (les commandes ci-dessus). J'ai ensuite mis en place une **chaîne CI/CD** avec **Cloud Build**, connectée au dépôt GitHub : à chaque `git push` sur `main`, Cloud Build reconstruit l'image (image simple, sans attestation), la pousse dans Artifact Registry et redéploie automatiquement sur Cloud Run. Je ne tape plus aucune commande : je pousse mon code, et la nouvelle version est en ligne quelques minutes plus tard.

Phrase à dire à l'oral

« Pour ne plus déployer à la main, j'ai mis en place un pipeline CI/CD avec Cloud Build connecté à GitHub : à chaque push sur main, l'image est reconstruite, poussée dans Artifact Registry et déployée sur Cloud Run, automatiquement. J'ai donc la chaîne complète : conteneurisation, registre, déploiement continu. »

9.5. Un arbitrage assumé : API directe plutôt que Vertex AI

J'ai **évalué** le passage à **Vertex AI** (Claude via Model Garden) pour rester 100 % GCP-native. La bascule de code est triviale (client **AnthropicVertex**, même identifiant de modèle, authentification par compte de service au lieu d'une clé). Mais Claude y est un **produit partenaire, non activable avec un compte de facturation en essai gratuit** : il faut un compte payant, et les crédits d'essai ne couvrent pas les produits partenaires. Pour ce test, j'ai donc conservé l'**API Anthropic directe** (fonctionnellement identique) et documenté Vertex comme évolution prête.

Phrase à dire à l'oral

« J'ai regardé Vertex AI pour rester 100 % GCP, et le code aurait à peine changé. Mais Claude y est un produit partenaire, indisponible en essai gratuit : ça impose un compte payant. Pour ce test j'ai gardé l'API directe, identique côté résultat, en documentant Vertex comme évolution prête. C'est un arbitrage coût/valeur assumé. »